Today

Welcome to your next lesson: Inheritance: sometimes kids are not like their parents.

I'm getting your lesson ready based on what you've already learned, what’s coming up, and the best ways to help you practice.

Step by step, you’re moving forward.

Imagine we have a general Pet that can speak, and then a more specific Dog that also can speak 🐶

Pet might have a speak() method that prints "Some generic pet sound".
Dog can define its own speak() method that prints "Woof!".

Same method name, different behavior for the child class — that’s the core idea of this lesson.

To start simple: if we have a Pet class and a Dog class that inherits from Pet, which would you expect to be true?

When I call speak() on a Pet object, I get the generic sound.
When I call speak() on a Dog object, I get a dog-specific sound.

Does that expectation feel natural to you so far?

Yes, that makes sense

Nice 👍

Let’s make it concrete with a tiny example. I’ll show you the idea, then we’ll play with predictions.

Here’s the shape we’re going toward (don’t code yet, just look):

Python
```
# Parent class
class Pet:
    def speak(self):
        print("Some generic pet sound")

# Subclass
class Dog(Pet):
    def speak(self):
        """Dog-specific version of speak()."""
        print("Woof!")
```
Key idea in this picture:

Both classes have a method called speak.
Dog overrides speak: same name, new behavior for dogs.
Quick check before we run anything:

If I do this:

Python
```
pet = Pet()
dog = Dog()

pet.speak()
dog.speak()
```
Which of these do you expect (just in your head is fine):

Both lines print "Some generic pet sound"
Both lines print "Woof!"
First line is generic sound, second line is "Woof!"

Tell me your choice (1, 2, or 3), then we’ll test it in code.

3

Nice, 3 is exactly what we want here 🎯

Let’s see it in a full, runnable program so you can feel what overriding does.

In the code editor, type this program (or copy it in carefully), then hit Run (you don’t need to Submit yet):

Python

```
# Parent class
class Pet:
    def speak(self):
        print("Some generic pet sound")

# Subclass
class Dog(Pet):
    def speak(self):
        """Dog-specific version of speak()."""
        print("Woof!")

# Main program
pet = Pet()
dog = Dog()

pet.speak()   # What do you expect here?
dog.speak()   # What do you expect here?
```

Steps:

Put this in the editor.
Run it.
Look at the two lines of output.
Tell me what the actual output was (just the two lines), and we’ll connect that to the idea of “kids not like their parents.”

```
Some generic pet sound
Woof!
```

Perfect 👌

So now you’ve seen:

pet.speak() → Pet’s version → Some generic pet sound
dog.speak() → Dog’s overridden version → Woof!

One small concept now:

Method overriding = a subclass writes a method with the same name as the parent, to give different behavior for that subclass’s objects.

For Dog, we overrode speak() so dogs don’t use the generic sound anymore.

Quick understanding check (no tools yet, just your answer):

If we add a Cat subclass like this:

Python
```
class Cat(Pet):
    def speak(self):
        """Cat-specific version of speak()."""
        print("Meow!")
```
and then do:

Python
```
cat = Cat()
cat.speak()
```
What line do you expect to see in the output?

I think it prints Meow!

Exactly right — it would print:

Meow!
``` 🐱

That’s overriding in action again: same method name `speak()`, but `Cat` provides its *own* behavior, so Python uses `Cat`’s version for `cat` objects.

Now I want to zoom in on how Python decides which version to run, but we’ll keep it simple:

If the subclass (like Dog or Cat) has its own speak(), Python uses that.

If it doesn’t, Python “falls back” to the parent (Pet) version.
```

Make sense so far, or want me to show another tiny example before we move on to some prediction practice?

That makes sense

Here’s a visual to lock it in 🔍

On the left: Pet.speak() with the generic sound.

On the right: Dog.speak() and Cat.speak() with their own sounds.

Clicking each object shows which method actually runs.

Next, I want you to drive a bit. 🚗

We’ll do a small prediction exercise with Pet, Dog, and Cat.

Ready to try a quick prediction with the picture you see here?

Parent vs Subclass Method Overriding

Yes, let's try

Take your time with the question below 👇

Look at these three objects in your program: pet = Pet(), dog = Dog(), cat = Cat().

If you run this code, which lines will print the generic pet sound "Some generic pet sound"?

python
pet.speak()
dog.speak()
cat.speak()
(Choose all that apply.)

A.
Only pet.speak()

B.
pet.speak() and dog.speak()

C.
pet.speak() and cat.speak()

D.
All three lines use the generic sound

Nice job, Ron 🎉 You nailed that.

pet.speak() uses the parent Pet version → prints "Some generic pet sound".
dog.speak() and cat.speak() each have their own speak() methods in their subclasses → they print "Woof!" and "Meow!", not the generic sound.

So only the actual Pet object uses the generic behavior. Dog and Cat override it for their own objects. 💡

Next step: let’s switch to a fresh, non-pet example (like a Vehicle and a Car) so you can see the same idea in a different setting. Ready to try that?

Yes, let's try

Cool, let’s reuse the same idea in a new world: vehicles 🚗

Think of this structure:

Python
```
# Parent class
class Vehicle:
    def start(self):
        print("Vehicle starting...")

# Subclass
class Car(Vehicle):
    def start(self):
        """Car-specific way of starting."""
        print("Car engine starting with a key or button.")
```
Same pattern as pets:

Vehicle.start() → generic behavior
Car.start() → overridden, more specific behavior

So:

Python
v = Vehicle()
c = Car()

v.start() # uses Vehicle version
c.start() # uses Car version
Now I want you to run this yourself:

In the code editor, create Vehicle and Car exactly like above.
In the main program, make v = Vehicle() and c = Car().
Call v.start() and c.start() and Run (no need to Submit yet).

After you run it, tell me the two lines of output you see.

Vehicle starting...
Car engine starting with a key or button.

Perfect, that’s exactly what we expect 🎯

v.start() → uses Vehicle’s version → "Vehicle starting..."
c.start() → uses Car’s overridden version → "Car engine starting with a key or button."

So again: Python chooses the subclass method when the subclass has its own version.

Let’s do one last, small prediction to lock in which version runs.

Imagine we remove the start() method from Car, so we only have:

Python
```
class Vehicle:
    def start(self):
        print("Vehicle starting...")

class Car(Vehicle):
    pass
```
and we run:

Python
c = Car()
c.start()
What do you think prints now?

You can just answer in words, like:
“It will use Vehicle’s version and print ...” or “It will error” or something else.
it will use vehicleses version

Exactly right ✅

Because Car no longer has its own start():

Python looks at Car → no start() there
Then it goes up to Vehicle → finds start()
So c.start() uses Vehicle’s version and prints:
Text
Vehicle starting...
So the rule you’re using is spot on:

If the subclass overrides the method → use subclass version.

If it doesn’t → fall back to the parent version.

You’ve now:

Written/used an overridden method
Correctly predicted which version runs in different cases 🎯

Let’s finish with one clean check on that idea. Ready for a quick multiple-choice question?

Yes, let's do it

What do you think? Pick an answer below 👇

A Dog object overrides speak(). When dog.speak() runs, whose version is used?

A.
Only the parent Pet version

B.
The Dog version when it overrides the method

C.
Both versions automatically

D.
Neither version

You answered that perfectly, Ron 🎉

When dog.speak() runs and Dog overrides speak():

Python looks at the Dog class first.
It finds Dog’s own speak() there.
So it uses the Dog version, not the parent Pet version.

The parent method still exists, but for Dog objects, the overridden one takes over.

You’ve now:

Overridden methods in subclasses to change behavior.
Correctly predicted which implementation runs for parent vs subclass objects.

That’s everything for this lesson — really solid work today 💪

When you’re ready, you can jump into the next lesson and build on this.

How would you rate this lesso

```
# Subclass
class Dog(Pet):
    def speak(self):
        """Dog-specific version of speak()."""
        print("Woof!")

# Main program
pet = Pet()
dog = Dog()

pet.speak()   # What do you expect here?
dog.speak()   # What do you expect here

# Parent class
class Vehicle:
    def start(self):
        print("Vehicle starting...")

# Subclass
class Car(Vehicle):
    def start(self):
        """Car-specific way of starting."""
        print("Car engine starting with a key or button.")

v = Vehicle()
c = Car()

v.start()
c.start()

```